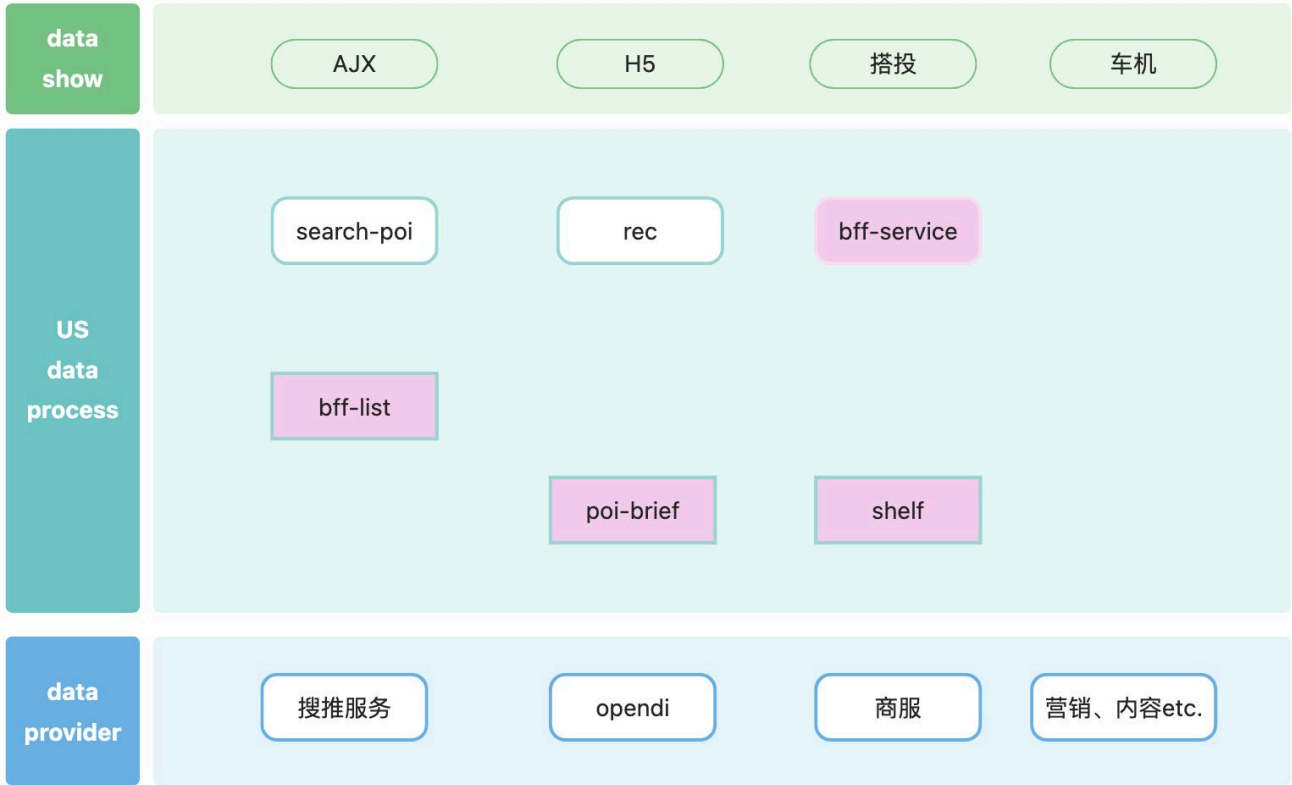


GBF在US建设标准业务能力

引言

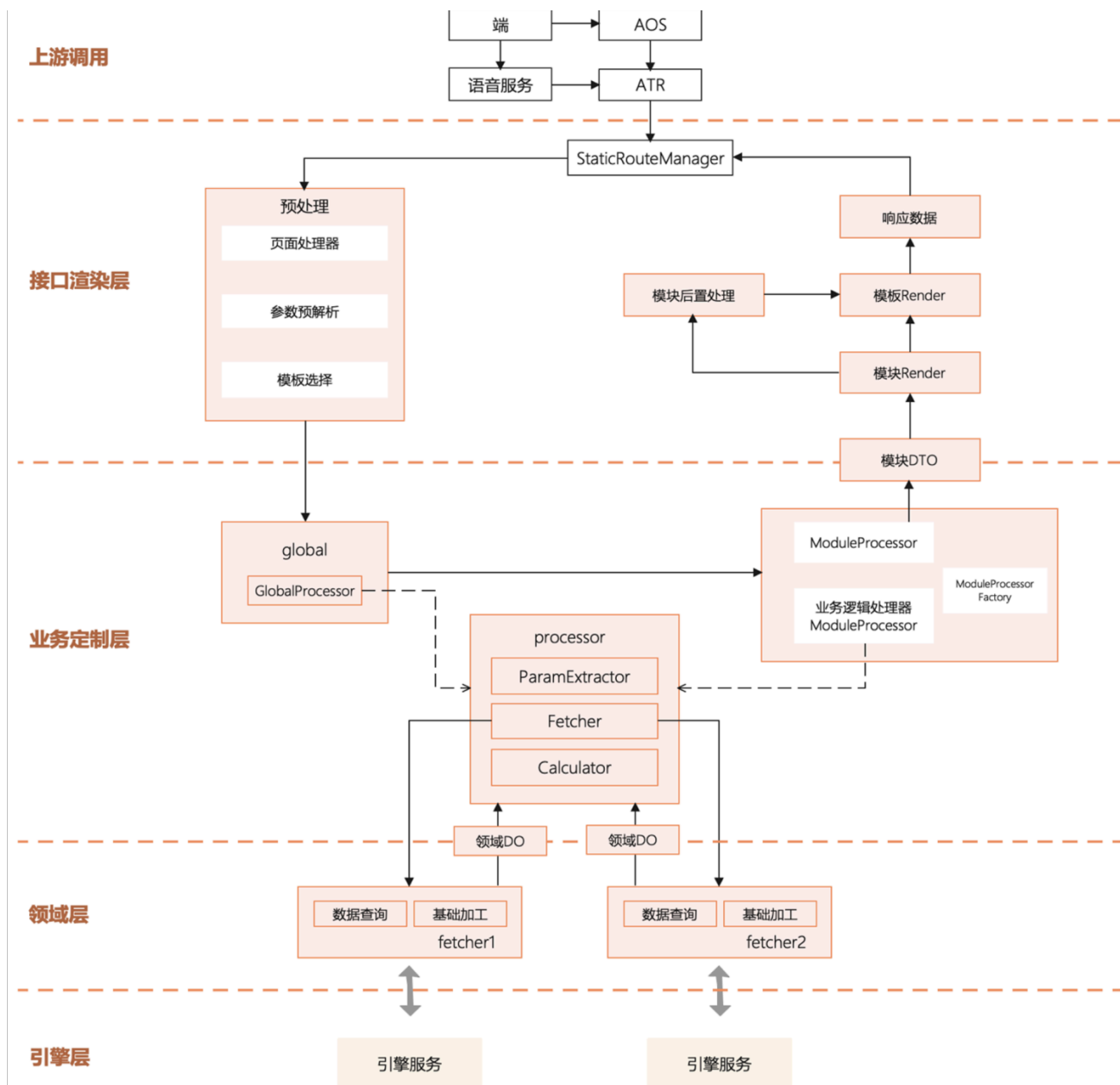
当下US的分层



search-poi、rec是早期us面向端开发的两个主要应用，随着迭代演进，在us内部按照poi、商品、通用功能等维度孵化出poi-brief、shelf、bff-list等应用，标志着us从面向端的 page、module、vo渲染的开发，开始沉淀出抽象、可复用的能力层。

MIF简介

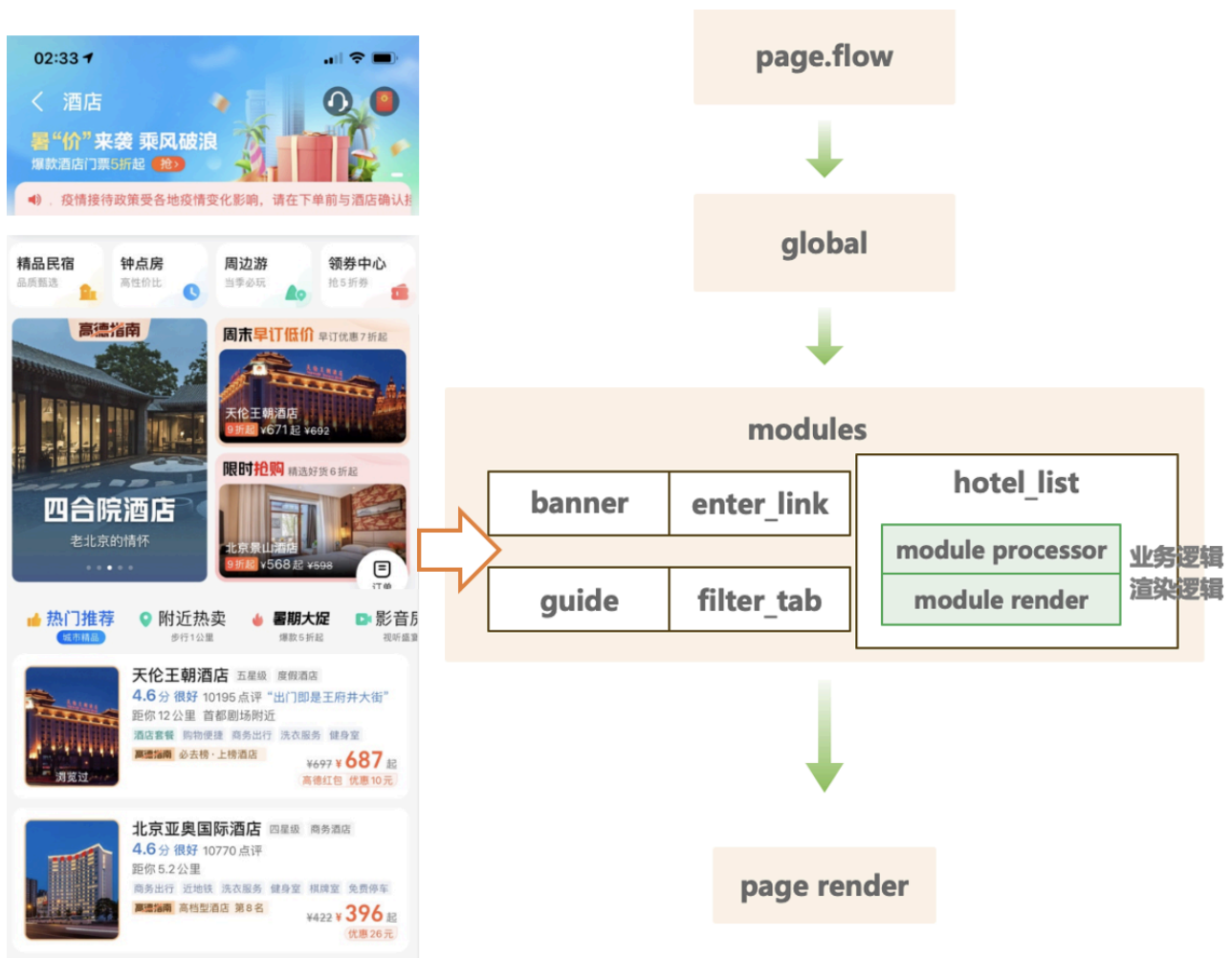
纵向架构分层



从上图的MIF开发流程结合业务迭代中的开发内容可以总结出经常改动的一些点：

- 新增页面时会从接口渲染层入手，结合业务定制层、下游领域服务层去做开发
- 当页面逻辑、下游域服务相对明确后，更多的是对模块的新增、优化逻辑，变更内容主要集中在业务定制层，即MIF模块的开发和迭代中。

横向模块拆解



以portal页举例，页面的横向划分如下：

- 页面级的接口注册
- 整个页面的flow编排，包括模块定义、排序、预处理等
- 页面级通用逻辑统一放到global进行管理
- 模块划分，banner、榜单、运营推荐位、一筛、feed流，每个模块相互隔离，输出渲染VO，交由flow编排
- 页面级别渲染，将每个模块输出的VO组合在一起，并对整体的结果进行封装，加工code、message接口级的泛化标准字段

思考



页面级设计

运行高效、模块复用

流程式调度

模块化数据

MIF的优势是基于页面级设计、流程式调度，向前端提供模块化的高效的数据服务。

随着业务逻辑持续的完善和复杂度的提升，不同行业的逻辑和设计也趋于更加明确和精细，us会进行更深层次的探索：在这种背景下，如何在不同行业的逻辑中以一种低成本的方式找到同质化的诉求，然后在这个共性的框架范围内高效的做好业务逻辑的精细支撑。

开发现状

酒旅的交易链路串联



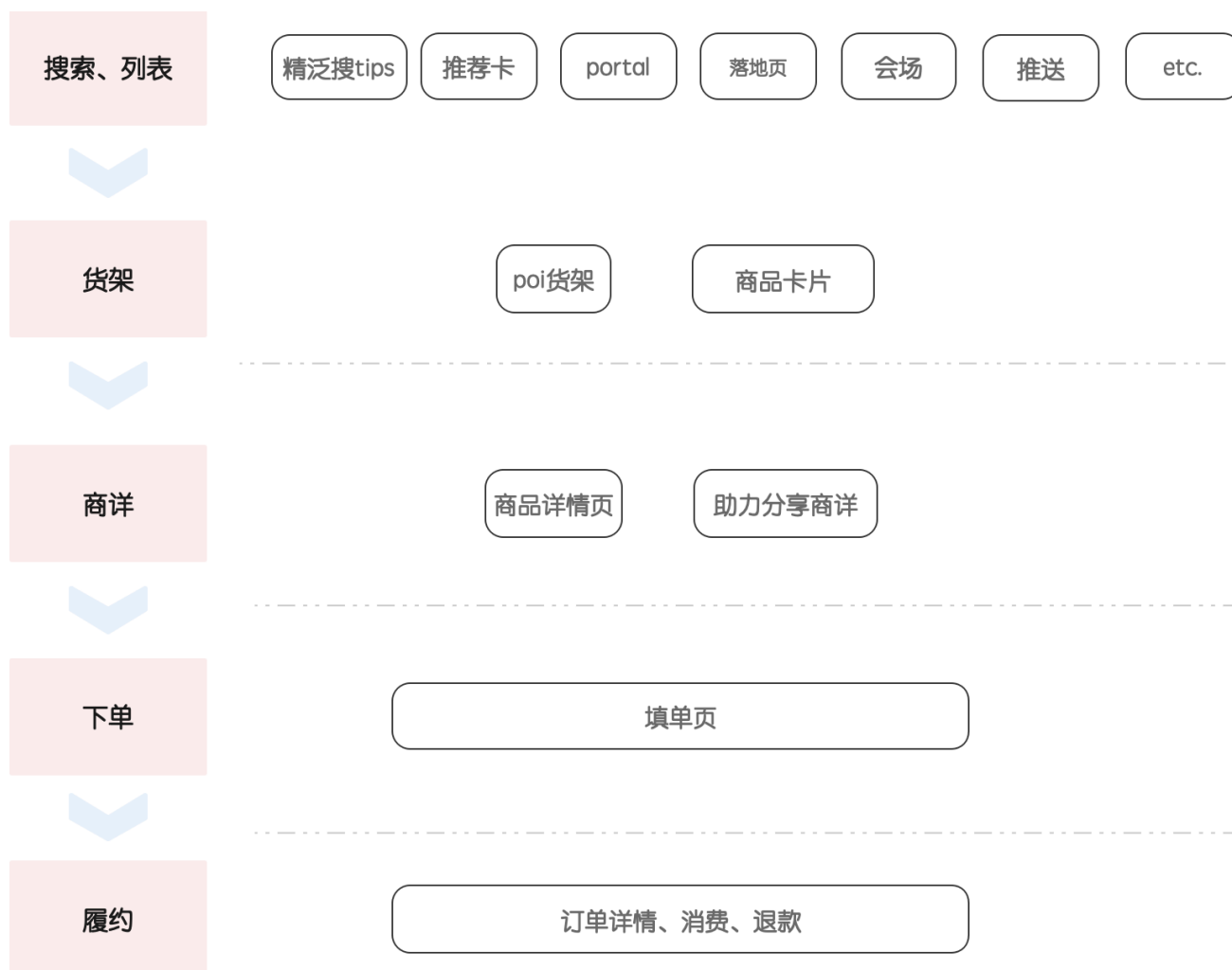
现状

绿色表示各行业统一

黄色表示各行业在各自迭代的同时，考虑到了模块的通用性，技术侧沉淀抽象出具体的复用能力。

红色代表各行业的独立迭代，不同行业间无关联。

从实际相对隔离的各垂类业务发展中，很容易会找到一些共性归纳出一套统一标准的全行业的导购交易链路



痛点

从上图可以看到，酒旅交易可以明确出一个链路：**入口展现-->货架-->商详-->填单-->下单履约**，参与商业化的其他行业也基本类似。那么为什么实际业务开发中并没有将系统按照该领域划分模块，而是在面对不同行业间的差异，标准链路在不同行业间隔离，将复用的难度加大。

背后的原因

每个行业的发展有先后，刚刚起步的行业和已经沉淀多年的行业对产研、生态的要求各不相同。

随着支撑范围的逐步扩张，各行业的提出的概念也越来越多，整体缺少统一的定义，导致跨行业的理解成本加剧。例如各行业的用户身份、营销玩法、场景落地、广告策略等，随着各业务的深入迭代，差异也越来越大。也导致了上述的痛点。

思考

先抛开实现，业务开发如何站在业务的角度去思考？

每个垂类虽然有各自的规划，方向、重点存在不一致，但通过更深入的思考抽象发现各行业都有一个共同的**路径和目标**：通过对人、货、场的灵活组合构建，从而让客户方便的参与交易、让商家提供优质的服务、让平台更精准、高效，从而促成更多的生意。

业务迭代的目标是持续不断的提高品效、坪效、人效。

人货场的最大意义是具备了形成统一的业务流程标识的特性，过去我们在谈了一个复杂的跨域的需求场景的时候，都是各个域的PD基于自己对场景的理解进行需求的分解的，而通过各个域对人货场的规则拉齐，就形成了跨域需求协同的一个有效标的物。

同时，借助标准组合形成统一业务流程标识，各个域通过多这个标识的记录，可以实现快速的基于统一流程标识的跨域业务分析、成本分摊等。



在标准化的业务语言基础上，可以针对不同行业需求做横向分析，剖析需求所需要的系统能力，然后进行关联域的识别，对领域能力的复用或补充完善由业务开发进行评估，确认后设计、编排、开发实现。

拿目前链路上完全隔离的两个行业的填单页举例

16:32



北京望京 798 和颐酒店

和颐大床房

房型详情 >

12月28日 (今天) - 12月29日 (明天) 共1晚

无早 | 1张大床 1.51米 | 有窗 | 27m² | 6-8层

12月28日 18:00 前取消扣取每间 32.22 元

查看规则 >

价格最低 好眼光！您已预订了该酒店最划算的房间！

整晚保留 酒店确认后房间可为您整晚保留

入住信息

房间数量

1间 每间最多住2人 >

入住人

王树伦

联系手机 +86

15210451961

预计到店

18:00 (房间将整晚保留) >

优惠 已为您选择最大优惠

高德红包

立抢 **省钱卡 90元红包** >

双旦特惠

立减 ¥6.44

省钱卡 | 有效期 30 天 · 随时可退 ⓘ

总价 ¥315.77
已优惠 ¥6.44

明细 ^

提交订单

14:04



珊瑚岛大冒险探险乐园门票三人票

全网低价 恭喜已享 9.9 折超值价！赶快下单吧~

随时退 携带身份证 无需换票 随买随用 [购买须知 >](#)

选择日期

今天
周五 12-30
¥288

周六 12-31
¥288

周日 01-01
¥288

更多 >
日期

选择数量

1

游客信息

 需添加 1 位游客，用于入园身份验证

王树伦

王树坤

许莲清

兴化

高德用户

新增/更换 >

高德用户

游客 请填写正确的证件号信息 编辑
请正确填写手机号

优惠明细

 已为您选择最大优惠

高德红包 **限时特惠满 140 减 2**



横向角度去看酒旅填单页主要包含 spu、sku信息，日期、优惠信息、客户信息、价格信息、会员信息

说明在业务在各自独立演进的进程中，用一套标准的业务架构作为承载是有可行性的。

GBF介绍

标准化业务语言是打破不同行业间壁垒的依据，业务的技术架构也需要清晰明确的架构语义对业务编排+开发实现进行设计和表达。

系统的发展历史进程由简单到复杂，具体逻辑的实现是由复杂拆解到简单。



先看当下从业务形态、产研协作上的合作方式：

首先有搜索的产品把控整个搜索推荐、基本信息展示这些内容，在这个基础上每个垂类都有各自的业务产品对行业的功能进行升级迭代，每个业务PM之间的大体方向是一致的，但是具体到细节上就会有千差万别的变化，也就是说业务产品在统一的顶层设计下聚焦自身垂类逻辑，并不关注其他行业的设计理念和逻辑细节。这些都是在技术侧统一拉齐认知的，但是技术侧大多是通过代码+设计文档去表达，coder之间的交流也大都依赖直接沟通和code show，code这种表达特别依赖 code as doc，如果做得不够精准或者表达不够方便清晰，会导致行业间相互理解的成本加剧，加大设计通用逻辑代码进行复用的难度。

所以就会发现一个比较有意思的现象：垂类业务的产品和技术之间经常会出现不一致，这种不一致包括理解层面和功能设计层面，就导致矛盾比较突出；相反搜索产品和技术之间的目标少有不一致的情况，就会显得比较和谐。

那么对这种情况US如何做进一步的革新去减少这种差异，首先就是US同学需要去思考业务，基于人货场的理论和提高人效、品效、坪效的目标，站在业务PM的角度上去看待遇到的矛盾和问题，从而帮助产品和技术减少矛盾，让产品的想法和设计合理落地。

思维模式的转变：自顶向下的渐序性 流程式设计 --> 自底向上的业务式 结构化思考

表达模式的转变：业务流程结合业务规则有明确的输出和表达。

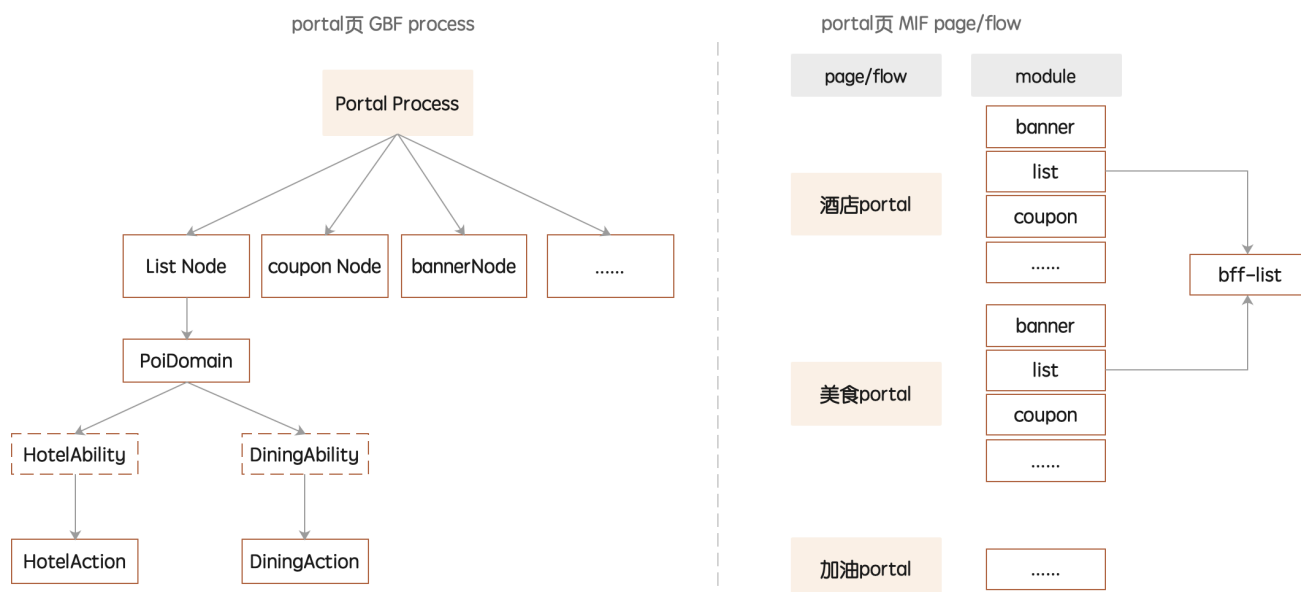
GBF概念剖析



1、Process

标准业务能力提供层，怎么定义标准业务能力？具有业务语义、明确目标的系统能力。站在us角度表达就是US面向上游端、平台、车机提供标准数据服务的能力。

GBF的标准业务能力通过NodeService的编排来实现。



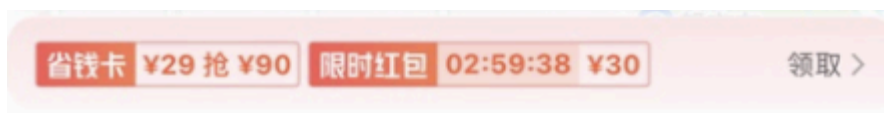
从上图左侧可以看出，portal页是一整个的标准业务能力，酒店、美食、景区等各个行业的portal是在这个基础上演进的，各行业间的差异性通过ability定义，action来实现。

回到MIF框架上看，各个行业的portal是相互隔离的，但是可以在这个相似模块上基于链路、代码这两个维度进行复用，不能很好的在系统设计中体现出场景和行业的层级和关联。

2、NodeService

节点或模块，每个模块之间相互独立，无调用和依赖关系，和mif的module比较像。

NodeService是形成标准业务能力的一个逻辑单元，是通过对不同域对象或方法的组合形成某种明确形态的对象，例如列表页中的优惠横条、优惠弹窗、优惠悬浮窗，三者从数据本质上看是一个：用户对应的红包卡券，所以是一个NodeService。然后在render层选择了不同的渲染方式，渲染表达的差异和红包自身逻辑解耦分层

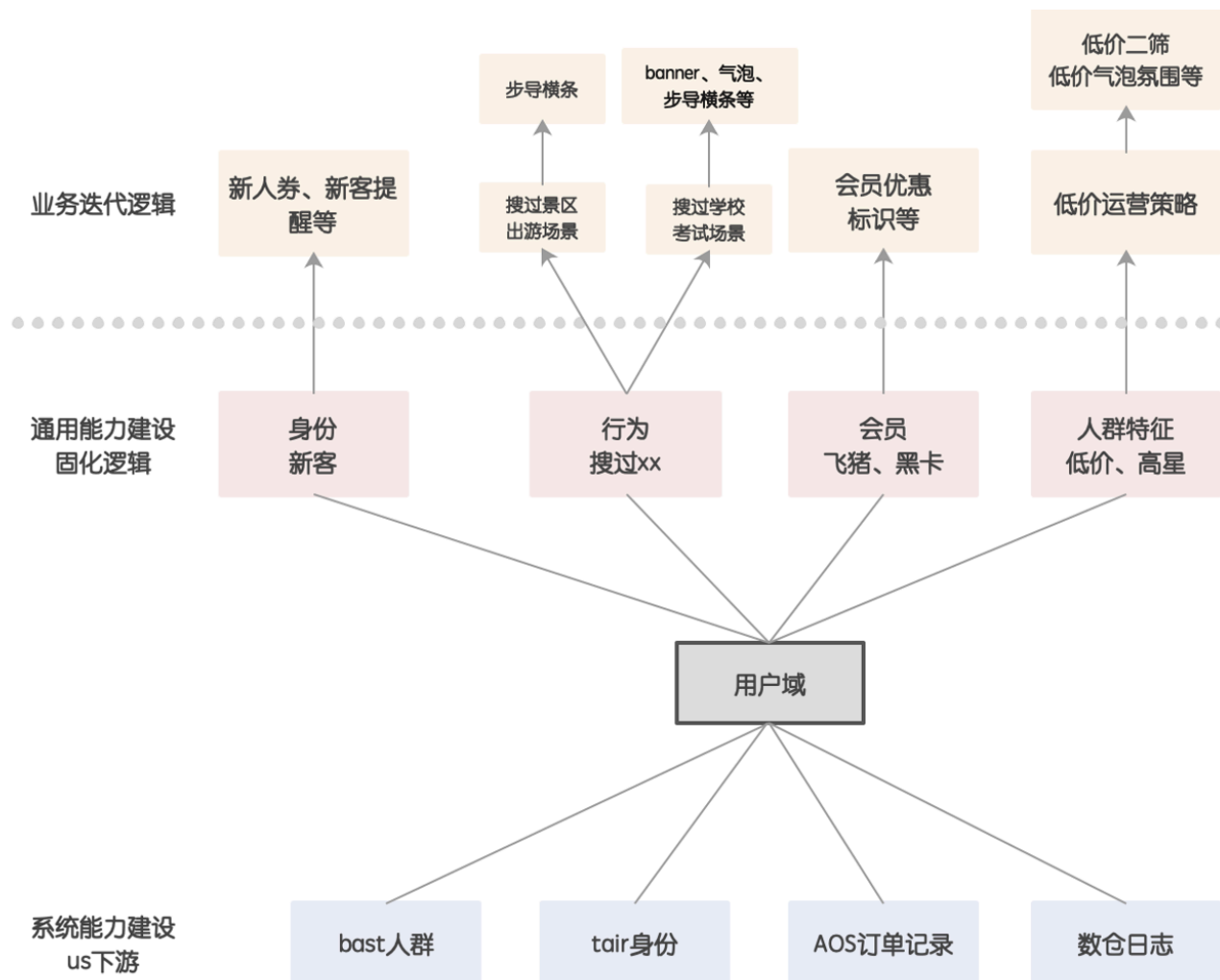


3、DomainService

领域方法的载体。这里并不代表整个领域，而是某个领域的方法的具体实现。例如 商品域获取商品基本信息，营销域获取秒杀活动信息，用户域识别会员身份等。

us需不需要域

需要域，按照自底向上的业务式 结构化思考这种方式做设计，需要提前定义好领域的划分。复杂问题拆解成简单逻辑的依据就是把问题拆解后放到个各自的领域当中去，然后再根据每个领域的功能、特性做出合理的设计，从而达到领域内能力维度的合理划分和复用。



拿用户域举例：产品某一期的需求迭代，需要识别用户行为（搜过学校考场）来判定考试场景，

当触发该场景时就要在搜索列表新增一个步导横条，初期平台建设us是没有场景识别的能力，那么在用户域就可以设计一个通用的场景识别能力：根据用户搜过的历史记录，判定poi的ttag，根据配置映射的ttag来判定是某个场景，产品的诉求是考试场景，但是功能设计上这个场景不仅仅局限于学校，而是可以根据ttag来设计N个场景，考试场景是其中的一个，也可以是出游场景、就医场景等，那么当这个用户域的场景识别能力建设完成后，就可以几乎0成本支撑类似的场景判定。

当场景触发时，新增一个步导横条，一期需求完成上线。

下一期需求产品提出：用户搜过医院，要触发旅游场景，然后在portal页展示对应的运营卡片或氛围，这个需求第一步可以直接借助已经沉淀好的用户域场景识别能力，然后直接进行下一步的开发。

综上可以总结，某个域的通用能力建设是相对固化的，基于通用能力的业务迭代逻辑是容易变化的，需求拆解设计时将两者解耦，就可以很方便的做到域能力的复用。

随着通用能力建设和业务迭代建设，可以将开发分为两类：平台研发和业务研发，进而将平台和业务从开发模式和人员配合上可以进行解耦。

如何划分域

us不同于下游服务专注做某个具体领域的拆解（商服、商家、离线、内容、搜推等），作为一个在信息业务承上启下的桥梁，us更多的是对各类服务的组织和聚合，对领域进行划分的方式方法会和其他下游有区别，us场景下的参照物也比较少，当GBF抽象概念定义完后，在us侧进行域的合理划分就会显得比较困难和模糊。

us既然作为聚合层，可以依据下游服务的粒度进行拆解，然后划分域。

先看us使用的各类服务如下



根据us使用的下游服务，按照业务的自然理解维度可以划分几类域



4、Ability、Action

Ability是定义扩展点和域的某个具体的能力，是一个接口， Action代表这个行为描述的具体实现。

ability并不是通用逻辑实现的地方，而是基于身份、场景命中后才会执行的特殊逻辑。

身份和场景：在us侧，身份大部分用来表示行业industry，场景则是在某个行业下更细分的一个维度，比如在酒店行业的portal页、泛搜列表页、附近页、详情页这些场景。也可以换一种角度看：泛搜列表页是身份，泛搜下的酒店、旅游、美食生服是场景。那么怎么决定哪个

维度是身份、哪个维度是场景，这个取决于实际case中，哪个维度更广，或者按照产品设计发展的过程中，先有什么，后有什么。

这些概念理解下来就会有一个感知：从API、SPI、PPI到GBF，意味着US技术同学从面向接口、面向页面编程转变成面向域对象、面向服务主题的编程。

GBF和MIF的区别

	GBF	MIF
语义管控	node、process、domain、ability、action、bizId、scenario	page、flow、module、processor
语义理解	比较抽象，需深入理解	简单直接
方法载体	service、action	processor、module、global
方法入参	Java bean	map
协议输出	HTTP、HSF	HTTP
代码及文档	有插件和UI支持	支持不完善
技术特性	传统阻塞式编程，使用ThreadLocal做context（暂时）	响应式编程，reactor IO多路复用，硬件利用率高
面向方向	面向域对象、服务主题、DTO	面向接口、页面、VO(DTO)

使用GBF进行领域划分和能力设计

1、列表页

澳洲康都

14:59

服务区

<

住 01-06
离 01-07

北京市

酒店

🔊

✕

位置距离 ▾

星级价格 ▾

推荐排序 ▾

更多筛选 ▾

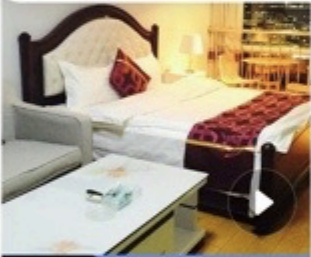
百亿补贴

品牌特惠

步行 10 分钟

买贵必赔

飞猪会员



4.9 非常棒

879 评价

距你 479 米 新世界百货 (望京店) 附近

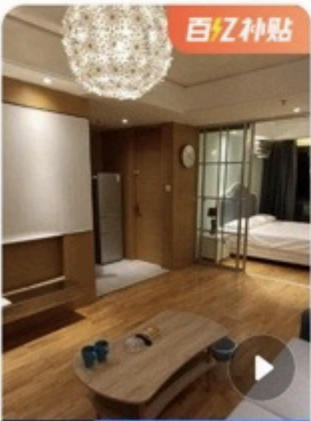
“登高远眺soho的好地方”

高德指南 青年爱住 第 2 名 影音房

¥242 **¥187** 起

高德红包 | 立减 | 优惠 55 ▾

特色 影音房观赛



源寓 (北京望京 SOHO 合生麒麟社店)

民宿

4.7 非常棒

39 评价

距你 500 米 望京 SOHO 附近

“前台小武服务特别周到”

高德菲住 近考点 近核酸点 影音房 行李寄存

¥421 **¥376** 起

百亿补贴 | 高德菲住首单优惠 | 优惠 45 ▾

特色 影音房观赛

买贵必赔 · 全网比价最高双份赔付

发现更多 >



距你 7.8 公里

安定门驿站 (北京...

¥149 ¥239 **抢**



距你 564 米

东篱居 (北京望京 ...

¥214 ¥329 **抢**



距你 566 米

源寓 (北京望京 so...

¥242 ¥362 **抢**



距你

伐木



北京清悦公寓 (麒麟社店)

民宿

4.8 非常棒

距你 514 米 望京 SOHO 附近

¥50 券



“浴室内提供拖鞋，24小时热水和吹风”
近核酸点 影音房 行李寄存 停车场 电梯



¥449 ¥427
百亿补贴 | 立减 |



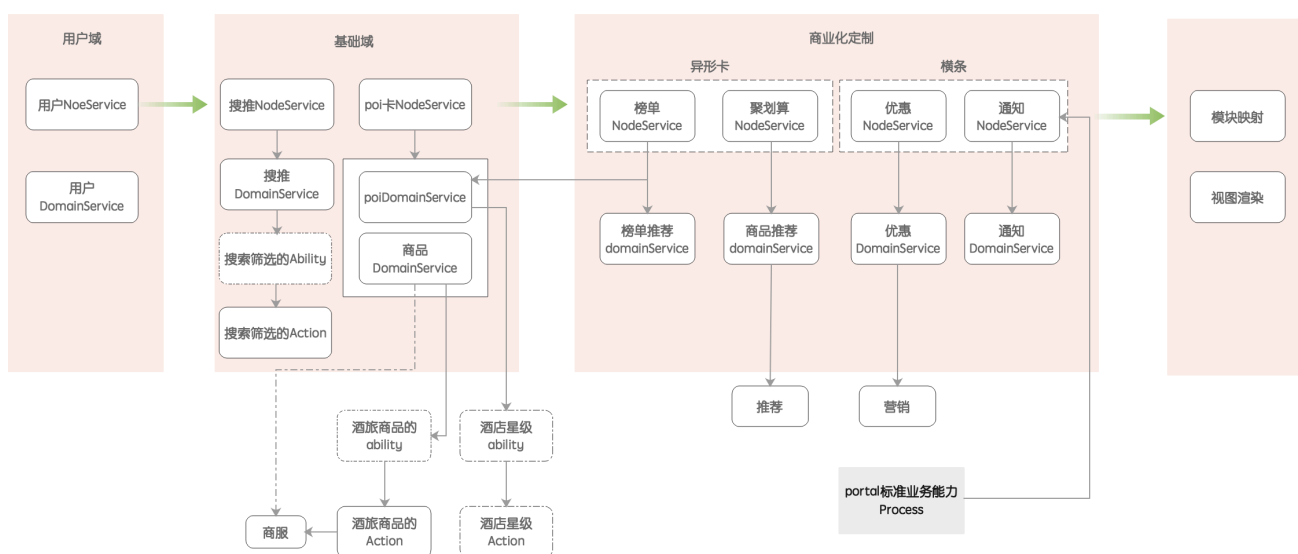
源寓 (北京望京 soho 悠乐汇店) 民宿

4.6 非常棒 15 评价

距你 566 米 新世界百货 (望京店) 附近

“银草很不错 空调也很好 干净整洁”

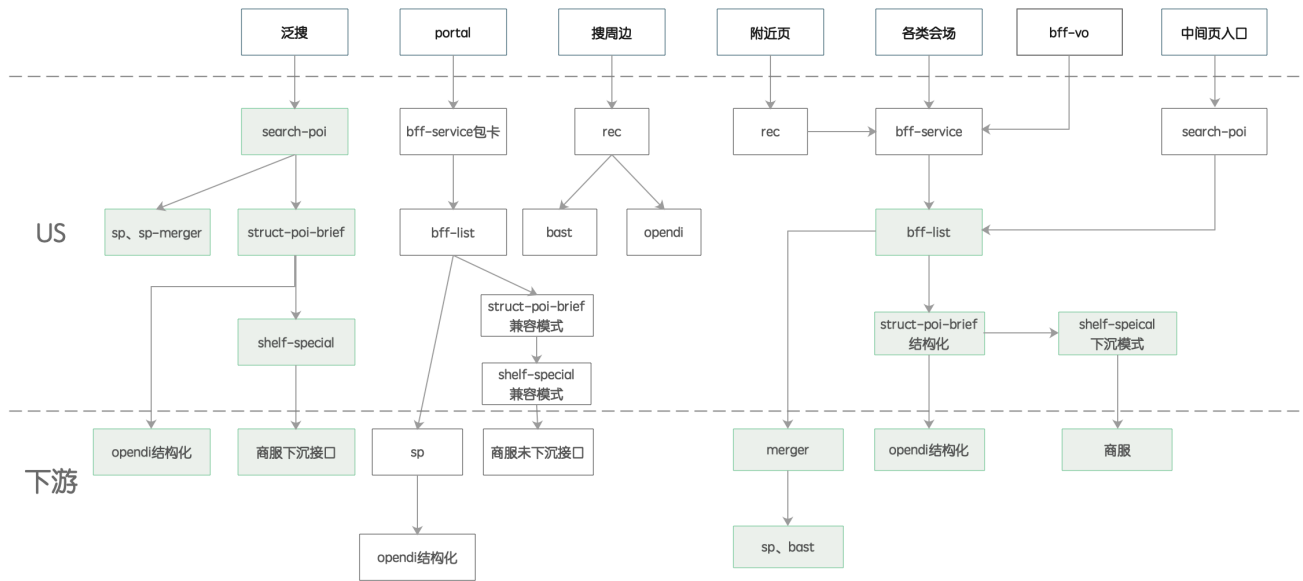
列表标准业务能力
Process



列表页的整体能力可以按照功能拆分为搜索支撑、横条、poi卡（包含货）、榜单、聚划算推荐、优惠等基础功能，其中筛选、poi卡从功能模块的角度来看是最基础的，和行业无关的，其他模块会根据各行业间的差异性决定出或者不出。

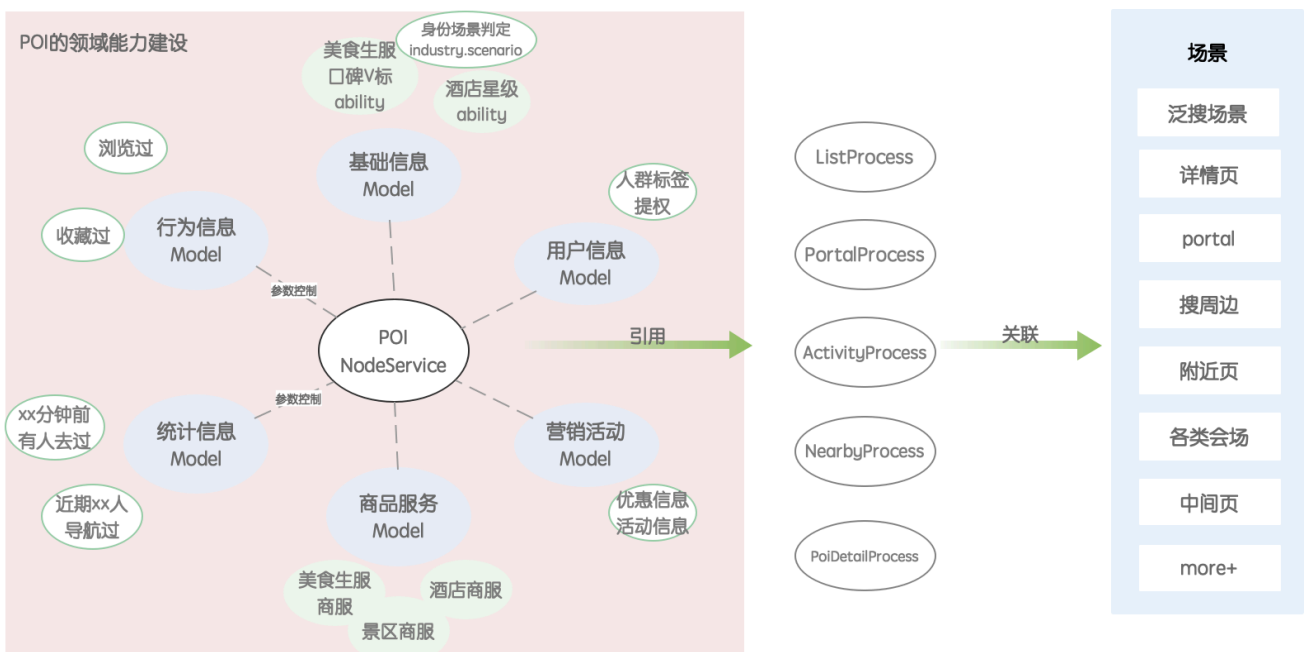
除了泛搜场景，portal feed流、附近页、会场这些场景同样会大量用到poi卡，也就意味着不同场景下，整个US都可以将该POI卡的逻辑收拢到一个地方，在支持差异化的同时，又能保证出处唯一、逻辑统一。

场景



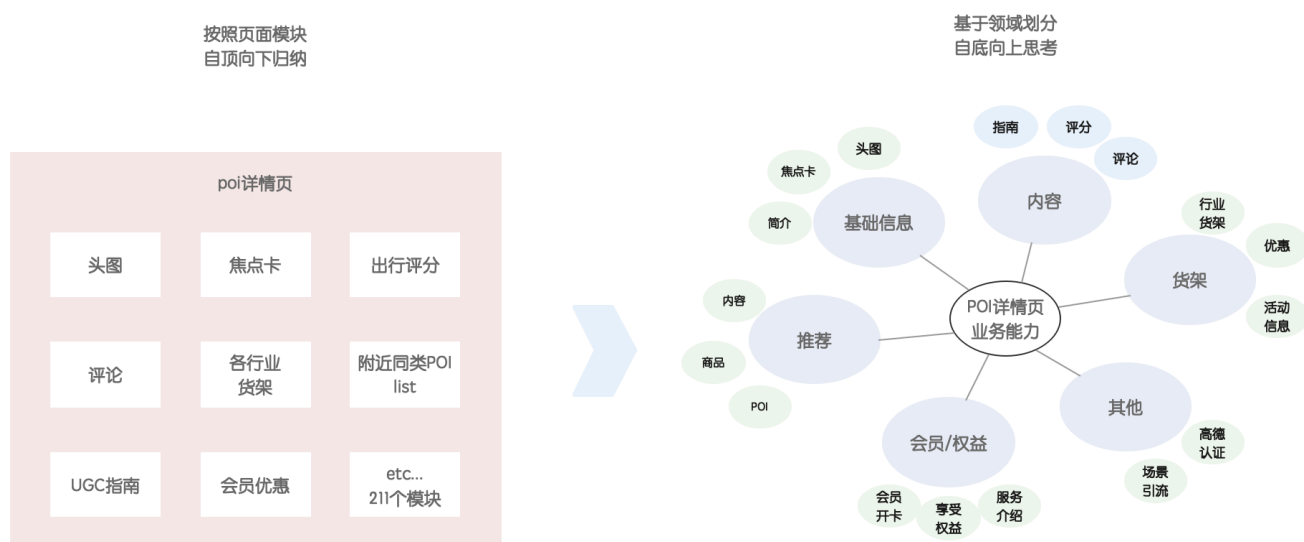
来看下现状：poi卡的加工逻辑在不同场景下是不一样的，当然这其中部分原因是存在历史原因导致的技术债务，上图绿色标注的可以理解为当下基于结构化沉淀的标准链路，在不同场景下poi的输出协议并没有完全一致：主搜poi是使用DO加工成VO，中间页链路是从DO转到DTO，然后直接输出DTO或由上游应用渲染VO。除了协议上的差别，链路上也稍有区别。

US以前抽象和复用的角度主要关注点在链路上，缺少领域的管理，容易导致当链路发生变化时，各领域间的调用和依赖关系发生调整，协议也容易发生变化（系统间解耦不彻底），不同应用之间出现逻辑相似度高但无法复用的代码。当抽象管理的维度添加领域对象后，拿POI卡这个例子看，可以归纳出如下的结构关系，借此可以明确各领域的数据结构表达和管理规范每个领域之间的行为。当POI NodeService明确后，标准业务能力层的Process可以直接引用，两者之间只剩下引用关系，不再有任何的关联关系。Process对应着不同的场景，自此POI的领域模型和方法对标准能力和场景解耦。

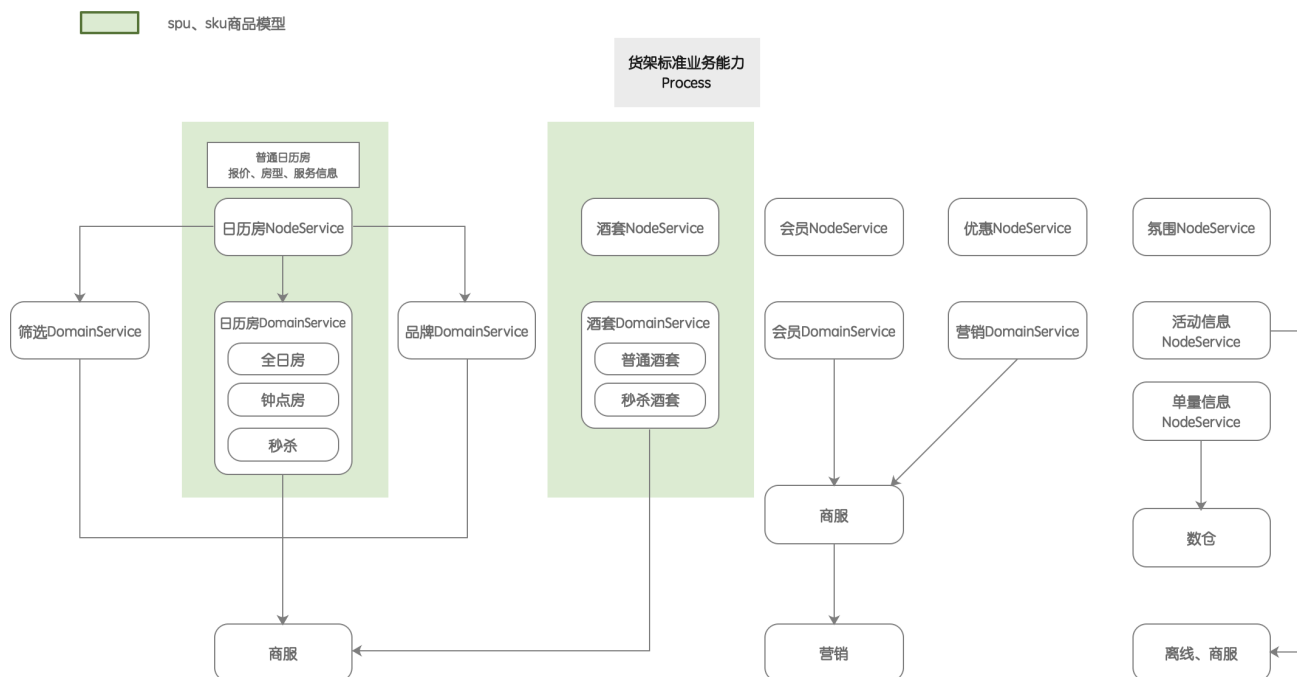


这种情况下会存在一个问题，在不同场景下，部分逻辑是可以不执行的，或者说存在某些场景需要执行特定逻辑，例如poi的泛搜场景需要展示浏览过、收藏过标签，其他场景不需展示。实现方式有两种：在PoiNodeService内部判断场景，根据场景判定要不要执行行为信息（浏览过、收藏过）的逻辑；还有一种是在场景调用时传入对应功能的参数，PoiNodeService按需加载执行。

当看完列表形态的poi使用领域模型解构后，再接着考虑详情页的领域划分，参照详情页结构化对模块的梳理成果[全行业模块现状梳理](#) 看到详情页的模块高达200多个，这么多的数量如果每个模块都要单独去维护对于有限的研发资源来说就显得特别紧张；当我们再去看对模块的分类可以发现数量不到10个，每个模块都可以找到各自的归属，也就意味着poi详情页的业务能力可以解构成10多个领域，比如基础信息领域可以支持的模块有20多个、内容支撑模块30多个、货架类模块50多个，内容领域还可以在进行细分（评论、流量、攻略、问答、通知等），按照这种思路进行系统设计，整体的业务架构就会变得清晰明了，业务开发同学也能够快速的分门别类定位代码了解逻辑，较低成本的找到关注点，当无复用逻辑新作实现时又能够确定放在合适的位置，这种业务和业务解耦能够帮助研发聚焦关联逻辑，同时也能帮助其对POI详情页的全貌有一定的感知，从而为分析需求的价值和合理性提供依据，也能够站在更高的层次上思考业务。



2、货架



上图中绿色框代表货架中表达商品的spu、sku模型，其实不难发现，每个行业的货架上陈列的商品（日历房、酒套、门票、美食生服券），都可以基于spu、sku的理念设计，所以换个角度讲，不同行业、不同形态的商品本身可以用一套spu/sku模型表达，各行业在这个基础上进行差异化。

那为什么现状是不同行业和形态的商品有不同的数据模型甚至服务链路？我认为是历史包袱和各行业商业化发展步调不一致的原因导致的，目前商品侧也在努力往统一的spu、sku模型上聚焦，也就意味着当商品模型使用统一的spu/sku模型语言时，上图中绿色标注的不同形态的商品可以合并到一起，从而使**不同商品的形态不再影响数据模型、系统链路的设计。**从理论上来看，商品形态是商品自身的一个维度，不该影响模型本身和系统链路。

18:06



< 汉庭酒店(北京望京... 找相似 写评价 反馈

省钱卡 ¥29 抢 ¥90

领取 >

01月12日 今天入住 01月13日 明天离店 共1晚 >

今夜特价

单早

双早

大床房

双床房

筛选 >

全网比价·享低价

高德系酒店类 100+
取价标准向你推荐



超值低价

双床房 A >

无早餐 双床 无窗 15m² 4层

不可取消 立即确认

高德酒店商家 高德全程服...

¥353
¥335

仅剩1间

预订

在线付

今夜特价 剩05:52:05 | 优惠18 >



性价比高

高级大床房 A >

无早餐 大床 无窗 18m² 4层

不可取消

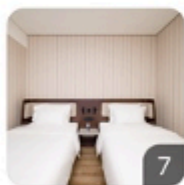
高德酒店商家 高德全程服...

¥383

仅剩1间

预订

在线付



双床房 A

双床 18m² 无窗

¥335
优惠18 起

无早餐 双床 低价 >

不可取消 立即确认

高德酒店商家 高德全程服务保障

¥353
¥335

仅剩1间

预订

今夜特价 剩05:52:05 | 优惠18 >

在

订单

无早餐 双床 >

Q
周边

☆
71

↻
分享

🚗
打车

导航

路线

11:07



< 颐和园

写评价 赢¥10

客服

反馈 官方

景区攻略



可订今日
立即入园

随时退
7天内到账

极速出票
5分钟内出票

官方门票 | 极速出票 入园保障 退款无忧

门票

已售 3.2 万+ | 首道门票，不含园内小景点

成人票

可订明日 | 有条件退 | 平均1分钟出票

已售 2.4 万+ | 购买须知 >

官方售票

¥20起

买省钱卡 共减1

预订

学生票

可订明日 | 有条件退 | 平均1分钟出票

已售 7000+ | 购买须知 >

官方售票

¥10起

预订

儿童

6周岁(含)以下或身高1.2米(含)以下

免费

老人/军人等 0元起 >

门票+皇家戏楼文化讲解

已售 700+



你有 4元 优惠即将失效 02:55:01

去买票



Q
周边

☆
收藏

↻
分享

🚗
打车

导航

路线

11:02



< 大地影院(望京新天... 找相似 写评价 反馈

优惠 评论 打卡 推荐

热映影片

可改签 >



满江红 淘票票评分 9.4

159分钟 | 悬疑、喜剧、剧情 | 沈腾、易烱千玺、张译、雷佳音、岳云...

今天 惠 明天 惠 周一 01-30 惠 周二 01-31 惠

11:40

国语 2D

14:19 散场

3号激光厅

37分钟后开场
¥69.9起
影城卡价 ¥67.9起

特惠

13:30

国语 2D

16:09 散场

4号激光厅

¥69.9起
影城卡价 ¥67.9起

特惠

14:35

国语 2D

17:14 散场

3号激光厅

¥69.9起
影城卡价 ¥67.9起

特惠

16:20

国语 2D

18:59 散场

4号激光厅

¥69.9起
影城卡价 ¥67.9起

特惠

17:30

国语 2D

20:09 散场

3号激光厅

¥69.9起
影城卡价 ¥67.9起

特惠

19:10

国语 2D

21:49 散场

换乘多步行累，打车单单有优惠

¥69.9 起
价 ¥67.9 起

特 订单

Q
周边

☆
508

↻
分享

🚗
打车

导航

路线

11:02



< 小吊梨汤 (凯德望京店) 找相似 写评价 赢¥10 反馈

朝阳区广顺北大街33号院215号楼05层501内05-01号

距你 444 米 步行 8 分钟



地图



电话

券 代金券

过期退 · 随时退

100元代金券

口碑 单次可用3张

¥92 ¥100

已售 1278 份

抢购

超值特价 · 全城热卖

过期退 · 随时退



招牌双人套餐

松仁爆三鲜 梨球果仁虾 小吊素什锦

随时退 免预约

¥176 ¥207 8.5折

已售 1166 份

抢购



4-6人超享套餐

酥香脱骨鸡 水煮牛肉 梨球果仁虾

随时退 免预约

¥405 ¥476 8.5折

已售 406 份

抢购



尝鲜双人套餐

干烧黄鱼 白灼罗马生菜 红油麻仁口水鸡

随时退 免预约

¥215 ¥253 8.5折

已售 121 份

抢购

¥20

¥40

¥50



当不同行业、形态的商品使用统一的spu/sku模型后，我们可以进一步思考货架的形态是否可以进行抽象，分析酒店、门票、电影、美食生服这些货架后可以总结出一个结论：从交互设计的角度上来看，货架存在三种形态



基于这三种形态，货架的模型也可以进行统一的设计和抽象。某个货架的形态并不是绝对的：在个别场景下是可以跳过其中一个环节，比如为了操作便捷日期会有默认当天的逻辑，当spu下只有一个sku时也会跳过spu的选择，无论是那种都可以从这三种形态中找到对应的设计。

当新增其他行业的货架时，就可以在这三种形态的基础上选一个，在设计复用的基础上提高代码复用。

3、填单页

16:32



北京望京 798 和颐酒店

和颐大床房

房型详情 >

12月28日 (今天) - 12月29日 (明天) 共1晚

无早 | 1张大床 1.51米 | 有窗 | 27m² | 6-8层

12月28日 18:00 前取消扣取每间 32.22 元

查看规则 >

价格最低 好眼光！您已预订了该酒店最划算的房间！

整晚保留 酒店确认后房间可为您整晚保留

入住信息

房间数量

1间 每间最多住2人 ▾

入住人

王树伦

联系手机 +86

15210451961

预计到店

18:00 (房间将整晚保留) >

优惠 已为您选择最大优惠

高德红包

立抢 **省钱卡** 90元红包 >

双旦特惠

立减 ¥6.44

省钱卡 | 有效期 30 天 · 随时可退 ⓘ

总价 ¥315.77
已优惠 ¥6.44

明细 ^

提交订单

14:04



珊瑚岛大冒险探险乐园门票三人票

全网低价 恭喜已享 9.9 折超值价！赶快下单吧~

随时退 携带身份证 无需换票 随买随用 [购买须知 >](#)

选择日期

今天

周五 12-30
¥288

周六 12-31
¥288

周日 01-01
¥288

更多 >
日期

选择数量

1

游客信息

需添加 1 位游客，用于入园身份验证

王树伦

王树坤

许莲清

兴化

高德用户

新增/更换 >

高德用户

游客 请填写正确的证件号信息 编辑
请正确填写手机号

优惠明细

已为您选择最大优惠

高德红包 **限时特惠满 140 减 2**

省钱卡 | 开通得 36 元 · 随时可退 ①

全场通用

总价 ¥286

已优惠 ¥2

明细 ^

立即支付



确认订单



满江红

共2张 原价 ¥ 159.8

今天 01-28 11:25~14:04 (国语 2D)

1号厅 3排6座 3排7座

奥兰环球影城 (望京方恒购物中心一店)

朝阳区阜通东大街6号院4号楼四层4-010

14:50

⊗ 不支持退票 ⊗ 超出改签时间[退改签须知 >](#)

本单可享优惠

优惠券&活动

已选最大优惠 -¥20 >开卡后, 本单立省6元 VIP

奥兰环球影城影城卡

¥9.9/季度 ☐

15210451961

手机号仅用于生成订单



购票须知

- 1.请确认场次和时间无误, 购买成功后将不予退换
- 2.由于设备故障等不可抗力因素, 存在少量场次取消的情

况，云进行退票退款

3.由于影院系统不稳定等因素，存在出票失败的情况，会进行退款

¥139.8

总计优惠20元

明细 ▲

立即付款



确认订单

 小吊梨汤 (凯德望京店)

招牌双人套餐

¥207

 随时退 · 过期退

购买数量



1



商品立减

立减 ¥ 31

优惠卡券

暂无可用优惠券 >

订单总价

¥ 176

 支付宝支付



 微信支付



购买须知

预约消费 无需预约，高峰时段可能需要等位

有效期 购买后 30 天内有效

购买限制 每人最多购买 1000 份

用餐人数 双人餐

使用人数 不限制

门店服务 支持免费 WiFi
支持开具电子发票

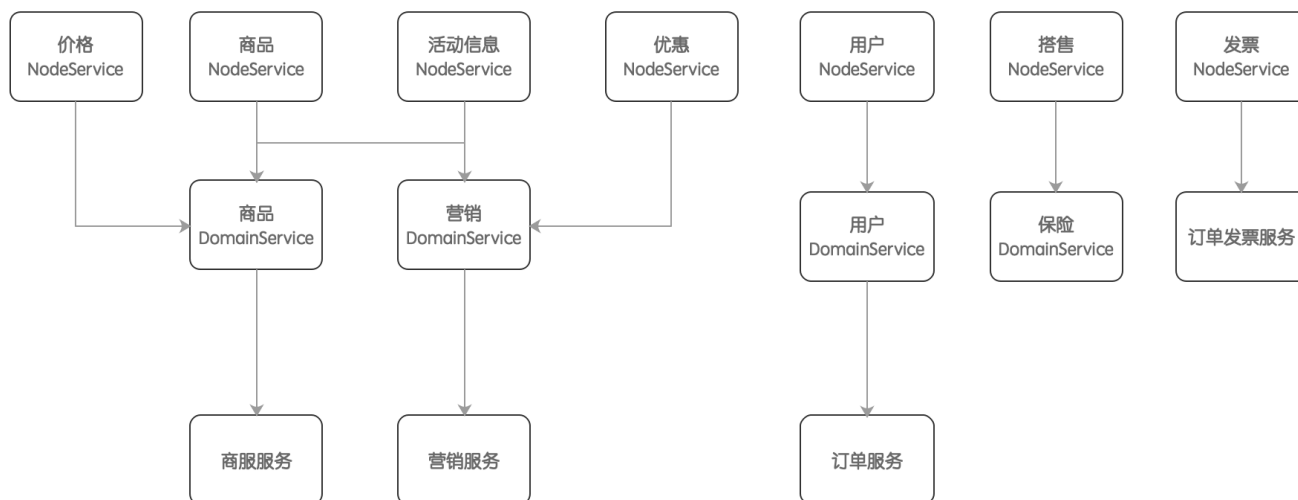
查看更多 >>

¥176

已优惠 ¥31

立即支付

填单页标准业务能力
Process

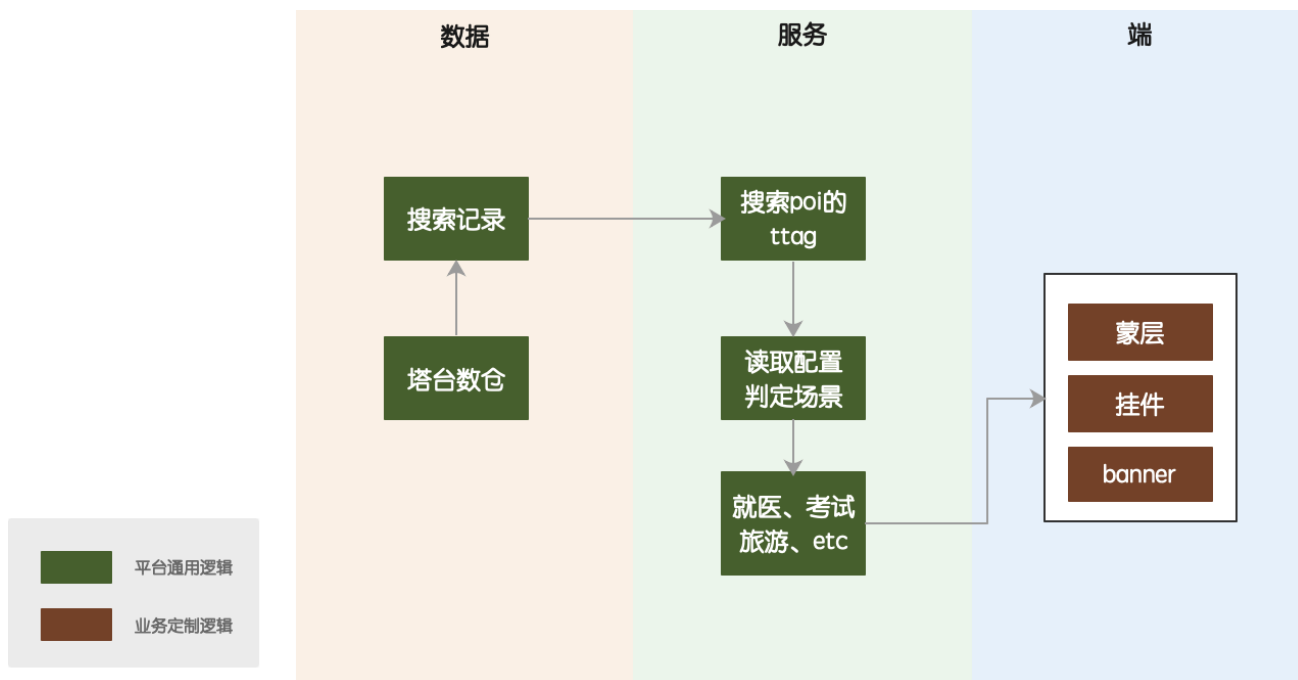


在货架中的商品模型统一后，该模型可以直接应用到填单页中的商品相关字段信息中。同时除了商品自身相关信息外，填单页还会有搭售（各类会员卡开卡服务）、联系人信息（根据历史订单自动带入）、比价（优惠和常规的比价明细）、发票等附加的信息，这些功能的信息又可以按照各自的领域进行划分和管控。其中发票服务是交由订单发票服务提供，us侧仅做注册。

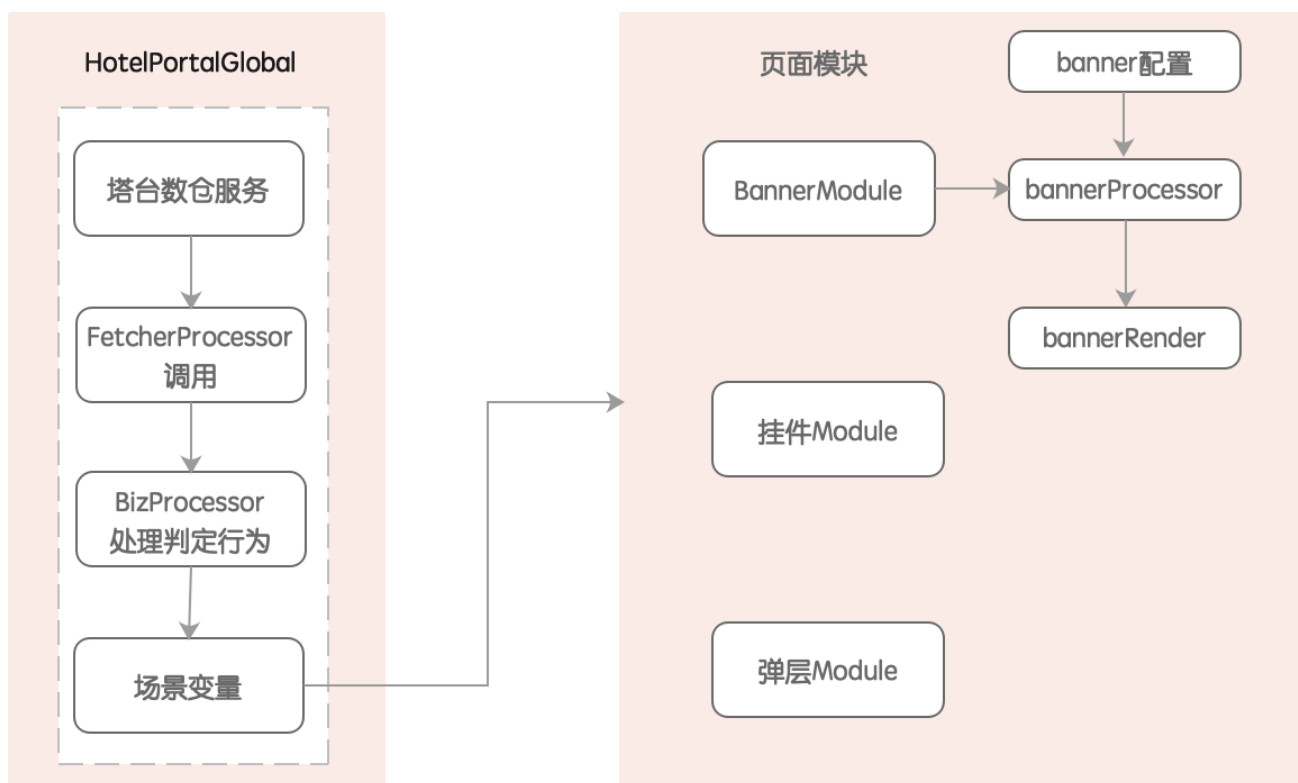
4、版迭需求case

2022.12月版迭 [酒店12月版酒店场景全链路营销资源位串联](#)

一句话概括需求：根据用户的搜索记录，判定用户的使用场景，根据使用场景，对酒店portal 页的模块（蒙层、挂件、banner、）进行干预。



那么使用MIF的时候是如何进行设计的？

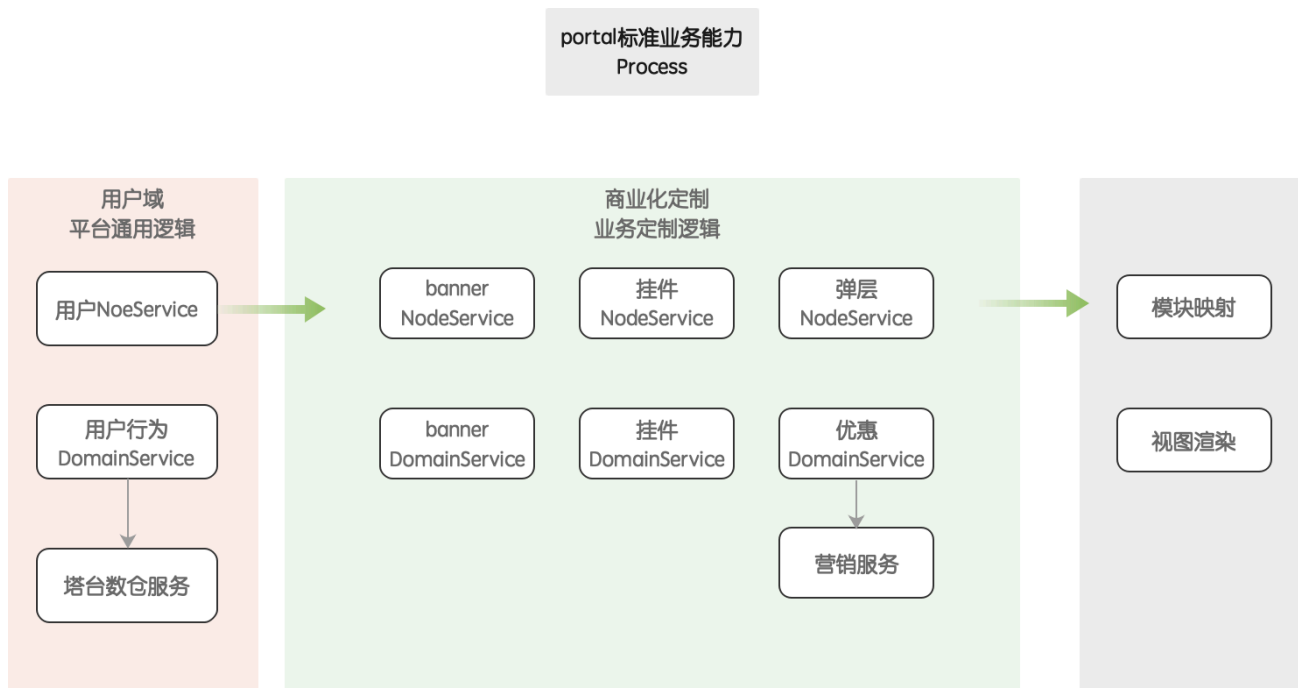


在portal页的global中加入场景判定的processor，然后生成一个全局变量，关联模块（挂件、banner、弹层）取该全局变量中的字段，然后各自根据配置加工渲染模块数据。这种情况下当列表页或其他场景也要使用的时候，如果在一个应用项目里，其他场景的global也需要重新声明场景判定的processor，喜欢抽象和代码管理规范的同学通常会建一个工厂类，创建该processor，然后在多个关联的global中声明；如果不在一个项目里，目前没有地方存放这类业务逻辑，就需要将该processor全部拷贝到对应的位置；如果不同行业的研发有相似逻辑但是没有提前打通的情况下，可能会各自实现，导致有两套代码。如果将这类业务逻辑放到二方包里供不同应用使用又会增加变更的成本来保证逻辑的一致性，从合理的角度看这类

平台业务代码应该只能来自一处，逻辑的更新也是一次更新各处生效，而不是通过jar包依赖的方式。

依照**具象需求点**转化为**抽象功能点**这一思路，我们换种角度来看这个需求的本质：

按照前面介绍的us侧领域划分，用户使用场景可以归类为用户域，再细一点就是用户的行为域，也就是说需求的前半段 判定逻辑是要在用户的DomainService里实现，和行业无关。当行为类型确定后，就是业务的定制逻辑了，无论是修改蒙层、挂件、banner亦或是其他页面、其他场景的逻辑，判定的依据都是一致的。换种角度讲，该需求需要使用用户的搜索记录判定行为类别，如果平台通用逻辑中无该功能，则需要在通用逻辑中进行设计添加，藉本次需求迭代使得通用业务平台具备这类功能后，后续就可以在其他场景或行业直接使用。



来看使用GBF的设计



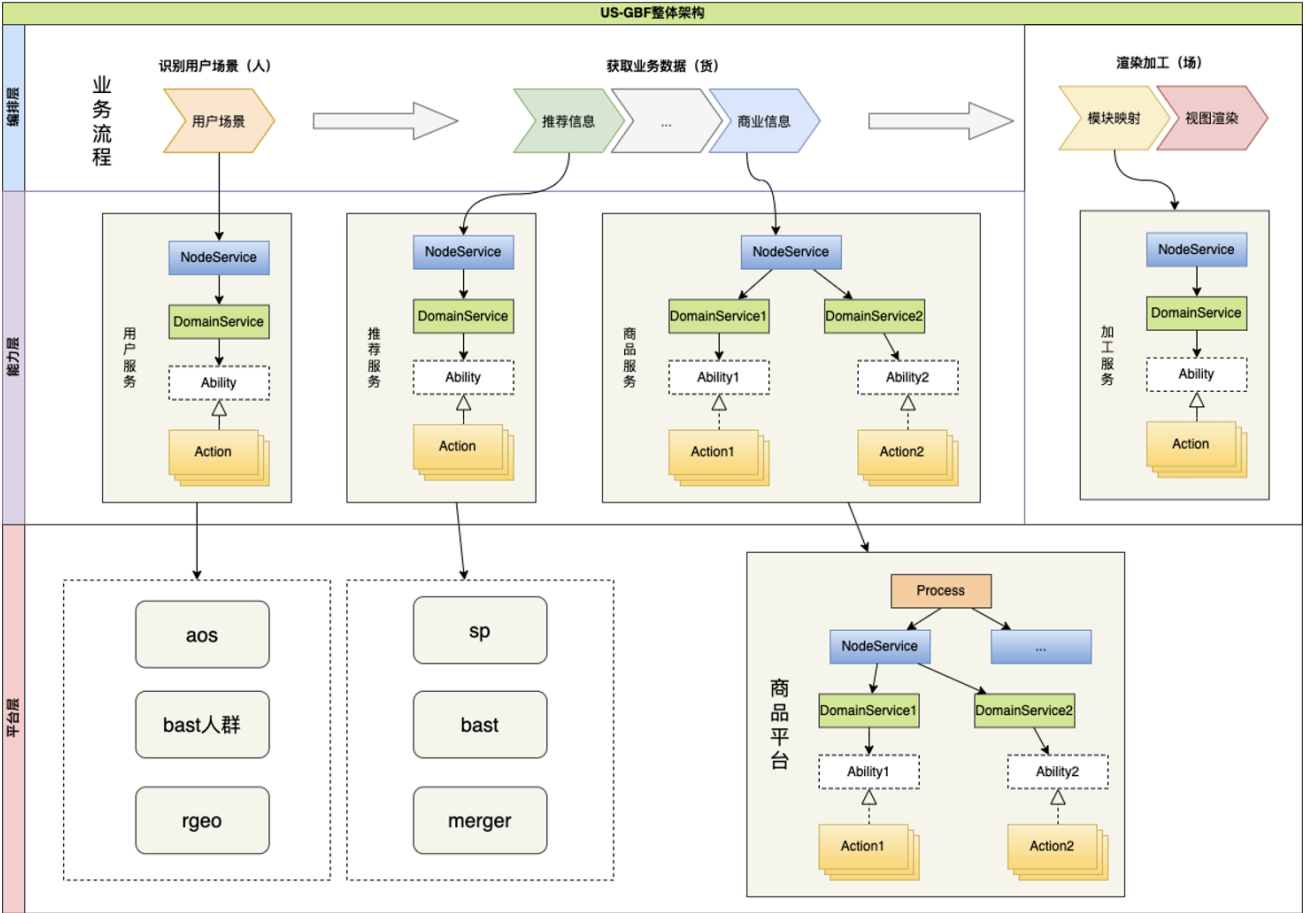
复制代码

1. 首先在用户域中使用用户行为的DomainService，如果没有则创建；
2. 然后调用塔台数仓，获取用户最近一次搜索的poi，并拿到poi的ttag(数仓没有就从poi域服务取，实际有)；
3. 再对ttag进行解析，此时需要一个将ttag映射成场景的配置，交由产品去配置划定范围并设置对应的场景关键字；
4. 根据配置得出的场景关键字和取值，根据业务的诉求，使用不同的NodeService进行干预处理；
5. 最后如果有渲染差异化的诉求，则连带输出场景关键字即可

从上面的步骤中可以看出，前三步是平台通用逻辑，和行业、场景无关，建成后可复用；后两步则是需求中具体到酒店行业、portal页场景的变更点。按照这种模式，将需求拆解为不变的通用逻辑和变化的业务场景逻辑，有了清晰的边界就有了解耦。这种情况意味着用户域的

细分内容（身份、行为、人群等）都有各自的领域进行管控，出处统一，其他场景如列表、详情使用时也是直接调用，无需重复声明，从而减少重复代码的可能性并降低复用的成本。

GBF在US的架构



US在GBF的分层

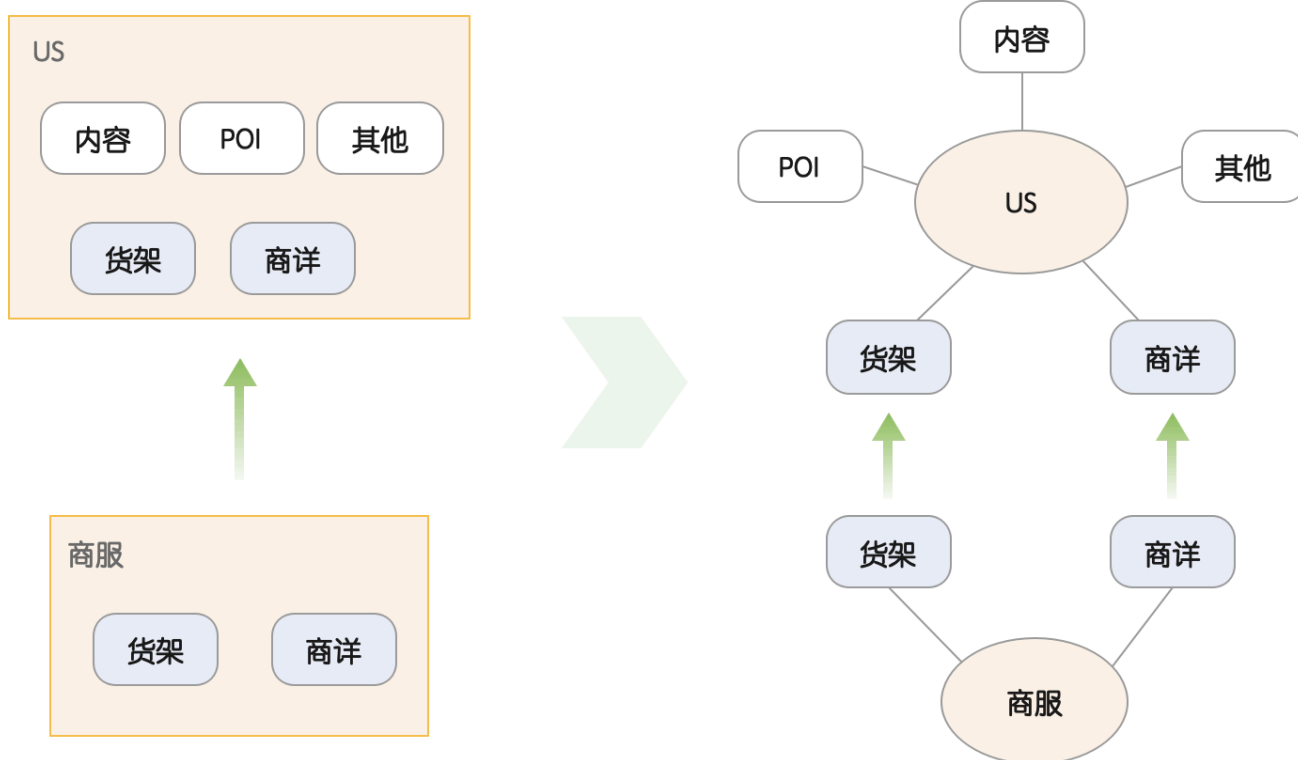


业务架构目标：框架结构清晰明确、数据协议标准统一、工具组件方便易用

领域和能力的建设，是需求来了后设计，还是提前考虑通用逻辑实现，在首次的设计阶段就要考虑好

GBF可以带来哪些收益

- 信息业务研发框架统一



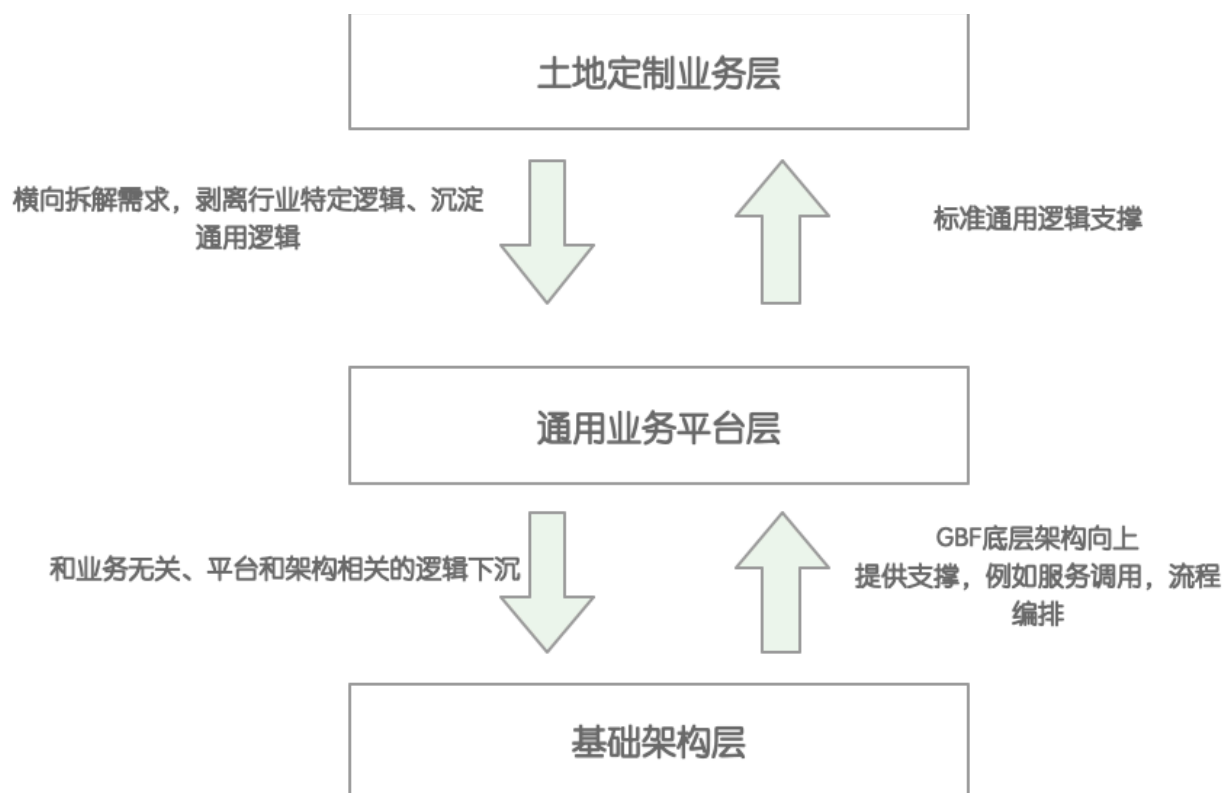
按照人员的组织结构划分，US和商服是两个不同的团队，业务的开发框架各不相同，不同团队之间的信息感知对齐要靠不断的沟通，在迭代和方案设计中存在分歧的时候需要耗费比较多的精力去达成一致。当研发框架统一后就会发现一个有意思的事情，US和商服的隔离就不再那么明显，两边的同学不只再关注自身内部的逻辑实现，同时能够更聚焦在商品相关领域的建设上，也就是目标和设计方案更容易对齐一致，减少差异和转化带来的内耗，从而降低团队间沟通的成本，提升团队协作效率。

再进一步思考一种可能：对商品模型和逻辑熟悉的同学A，可以打破US和商服团队的边界，从商品服务到给前端提供商品的标准模型数据，一个人或者小组内部就可以承担这个事情，让事情在人员组织上更聚焦，从领域模型的角度上来看商品相关的迭代也就变得更内聚。

- 领域模型定义
- 结构化的可视化表达

会听到有同学觉得us介入成本高，首先mif框架作为一个业务开发框架，存在学习成本（一次性的）之外，业务的复杂度和不同行业的差异性是非常重要的一个因素。只要不同行业一直存在差异，那么该边界就无法打破，但是可以借助清晰的结构定义和可视化流程表达来降低业务逻辑的理解成本

- 开发分层



按照这种设计理念，基础架构层提供技术相关的系统功能支撑，作为GBF的核心概念定义和流程编排设计，都在这层去实现，前期建设完成之后无需再进行建设。无论是GBF还是MIF或者其他开发框架，基础架构层是最基础的一层，同时提供方便易用的工具和组件也决定着业务开发人员的开发体感和效率。

业务平台层是抽象出来的和行业无关的逻辑层，业务平台层跟随信息业务发展的形态演进，例如召回的逻辑从sp/bast演进到merger时，业务平台层的设计也会随之发生变化，土地业务层和基础架构层则不会发生变化。当业务迭代演进时，先横向需求分析，剥离和行业无关的逻辑，例如前面提到的版迭需求case里，根据用户的搜索记录去判定用户的使用场景时，这就是很明显的业务平台层的逻辑，同时也不难发现，搜索的通用逻辑都可以归类为业务平台层，各垂类在这个基础上进行扩展和定制。

当定制超出搜索的边界时，如果定制对其他行业又有很强的适配性，那么也可以归类为平台业务的逻辑，例如酒店提出的品牌搜的banner定制诉求，特斯拉的品牌二筛诉求，不同行业的不同定制就是对特定品牌命中后，对不同模块的干预和影响。这些都可以借助土地定制的迭代，将其归类为通用业务逻辑，当美食行业对品牌迭代（例如给海底捞展示品牌的banner）时可以直接使用已存在的通用业务逻辑。由此可见，判断是否通用业务的标准是该业务逻辑是否能够剥离行业属性或对其他行业有没有适配性。

也正是基于这种边界的分层，将**业务和平台解耦**，将不同行业的差异隔离，**业务与业务解耦**。

• US做聚合，不做胶水

这是一个聊的比较久的话题，哪些事情US不适合做。因为个别应急的情况，或者us历史逻辑比较重，再或下游一些能力的缺失、关联方成本高等各种因素，US作为承上启下的一层，上下游的目光都会聚焦在US这里，颇有种“受命于危难之际奉命于危难之间”的景象，权衡成本

之后通常会选择临时方案、自行消化，长此以往US就做了很多胶水的逻辑。后续的修正和推进通常伴随着上下游理解不一致经常出现卡点，节奏就拉的比较长。

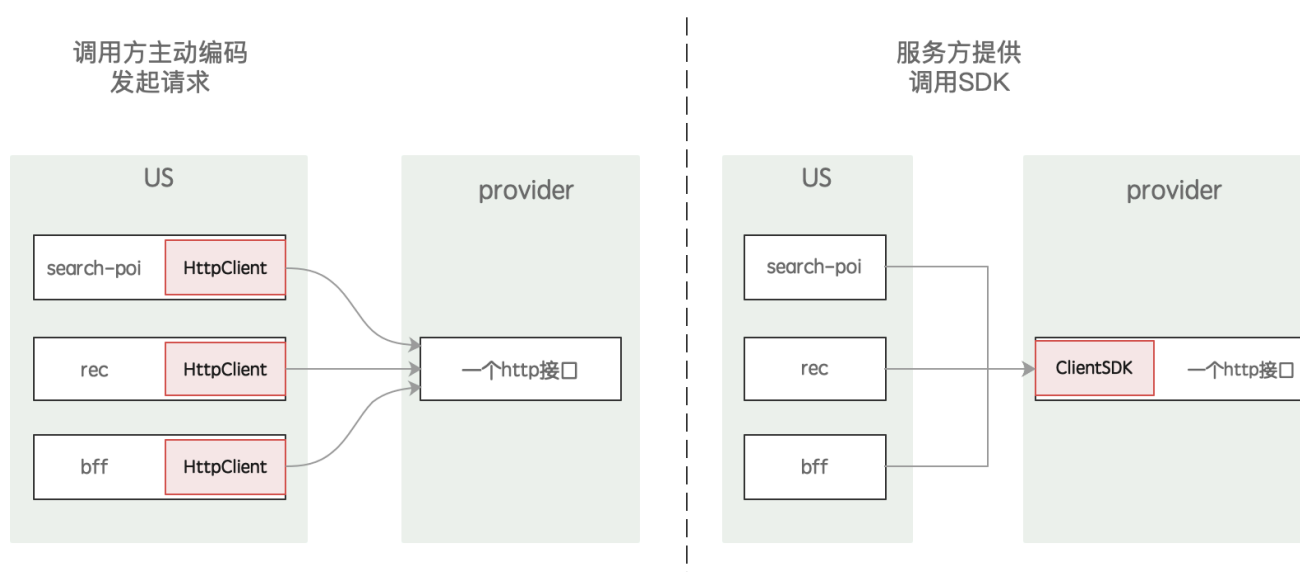
US借助GBF和领域模型对业务进行解构时，能够产生清晰的边界和脉络，这些作为US判定业务逻辑边界的依据时显得更加有章可循。胶水代码在不可避免的情况下则可以交由（bff-vo或其他渲染层）实现，从而保证领域内部服务逻辑的规整。

Feature

1、要聚合、不要胶水

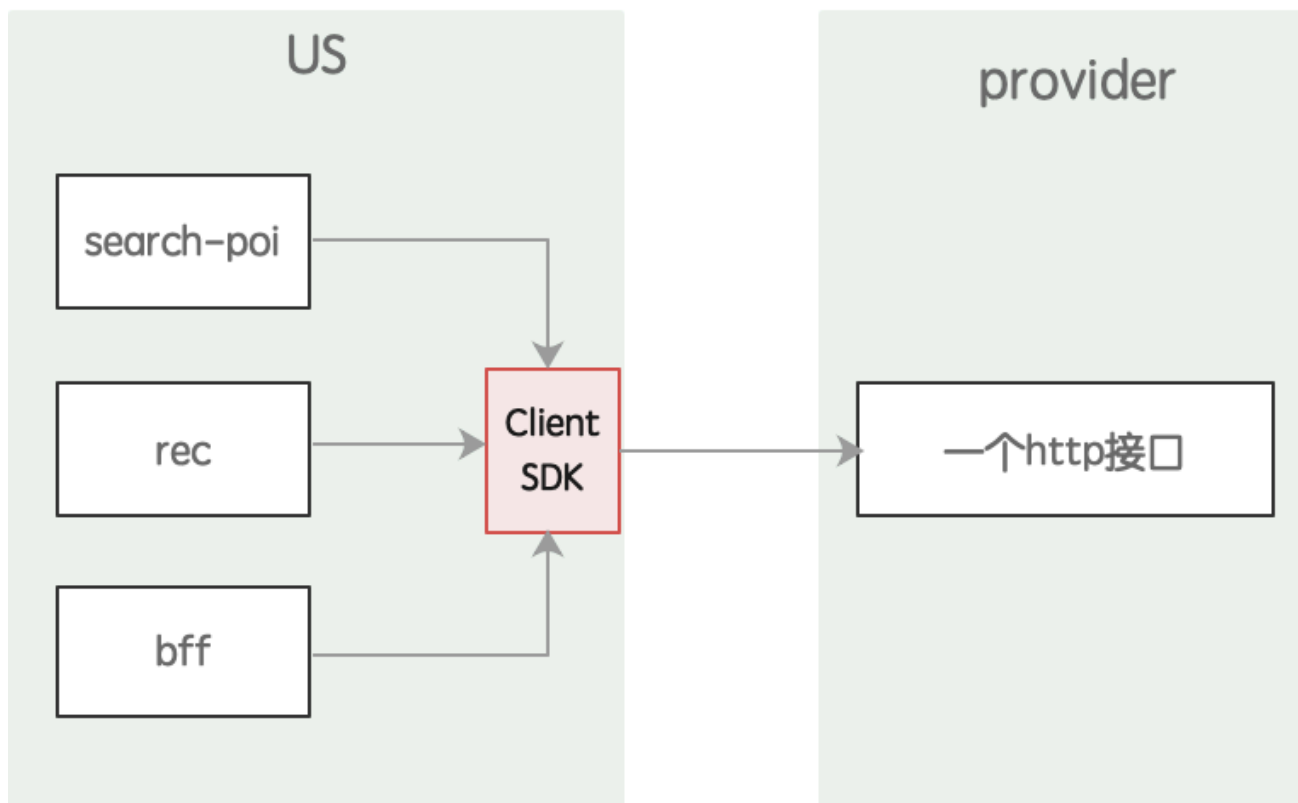
2、适配下游、调用方式思考

常见的两种调用方式



us对下游http接口的统一封装

调用方生成sdk 统一调用



在这个基础上，屏蔽http调用，抽象简化为一次普通的方法调用，即 入参-->执行-->输出 这个模式，按照这种方式就很方便做到http调用的复用

这么设计的一个宗旨是 重点关注领域划分、能力建设和业务逻辑的实现，不关注下游调用的方式类型以及差异。

3、去掉if/else，使用规则引擎

规则的表达



复制代码

```
return processor.runIf((Predicate<GlobalContext>)globalContext -> {  
    return  
    "3".equals(globalContext.getPreParsedInfo().getParam("use_userloc"));  
})
```

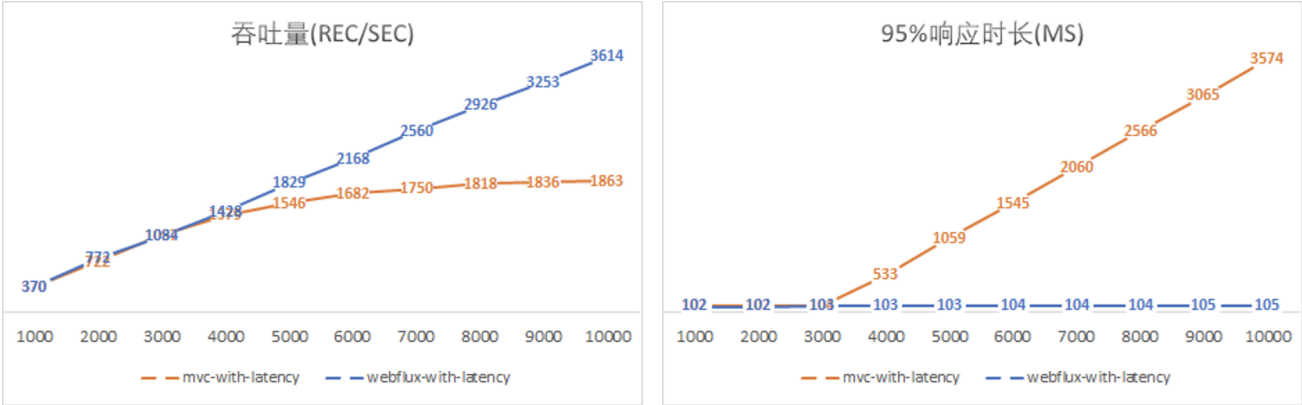


复制代码

```
@Condition(UserLocProcessorPredicate.class)
public void userLocProcess() {
    /**
     * 方法体实现
     */
}
```

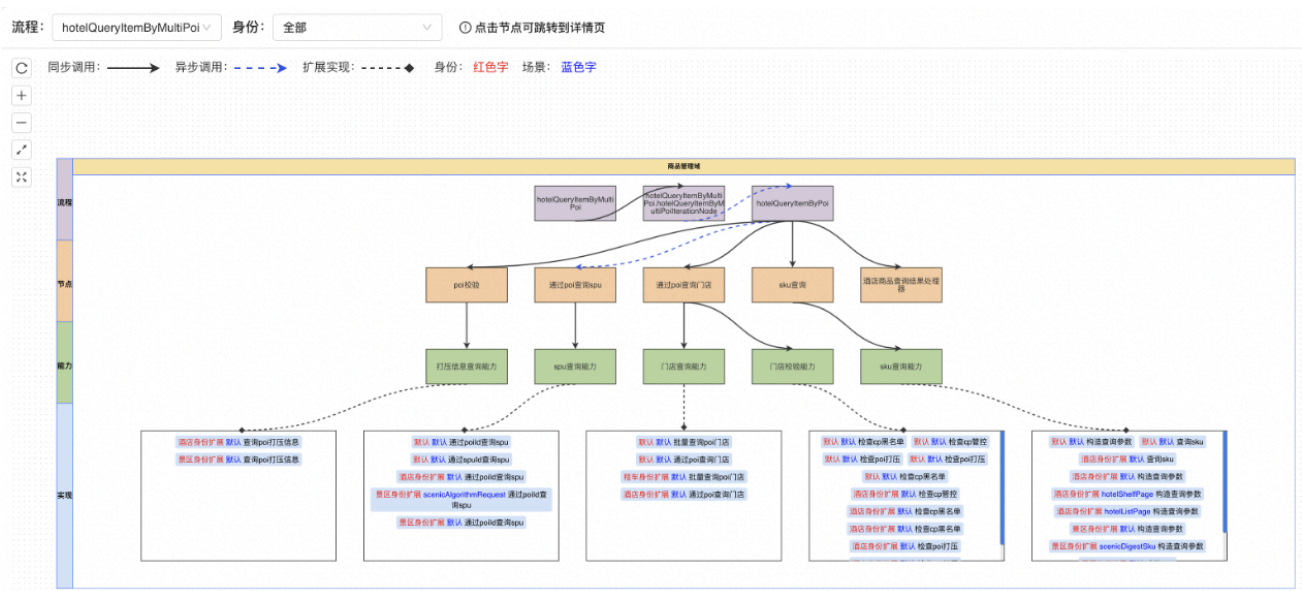
第二种方式更容易使用规则引擎，从结构上对流程的代码干扰更少，更容易对这类的实现进行抽取，从而接入可视化（业脉）

4、NIO优化



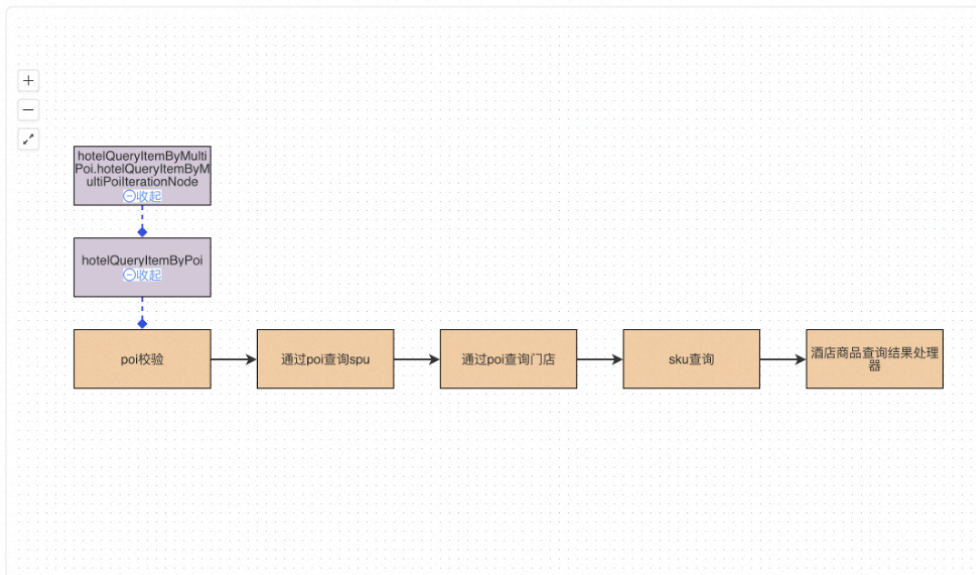
spring webflux(reactor) 或 Java19 虚拟线程(新特性预览)

5、可视化表达



基础信息

编码	hotelQueryItemByMultiPoi
名称	hotelQueryItemByMultiPoi
入参	
出参	
应用	gic-if
领域	商品管理域
创建时间	2022-09-28 09:33:51
更新时间	2023-01-28 11:15:08
描述	



做好code as document的基础上，通过对代码解析的方式表达业务逻辑，让开发同学能够最小成本快速了解逻辑。

参考资料

<https://yuque.antfin-inc.com/amap-search/gp80z4/czkl26?singleDoc#> 《票品详情接口》

<https://yuque.alibaba-inc.com/amap-search/gll2zw/uu1a2fb6400g8ady> 《酒旅业务功能梳理》

<https://yuque.antfin.com/amap-search/nen549/splhtbqbucx5elxf?singleDoc#> 《GBF使用手册》

<https://yuque.antfin.com/amap-search/nen549/tghr6aibad67cttx?singleDoc#> 《GBF分层设计及模型》

<https://ata.alibaba-inc.com/articles/246744?spm=ata.23639746.0.0.31f235657UJyZF> 《业务支撑视角下的人货场》

<https://yuque.antfin-inc.com/amap-search/tol7g9/lgtu2q?singleDoc#> 《能力介绍》

tiger代码库 https://code.alibaba-inc.com/tiger-shark/tiger-shark/tree/feature/20221207_14867675_commodity_platform_1/

星环入门学习最强指引: <https://ata.alibaba-inc.com/articles/143837> TMF&星环学习资料汇总（持续更新）: <https://ata.alibaba-inc.com/articles/132543> 星环 - 产品整体概述:

<https://ata.alibaba-inc.com/articles/108514> TMF总体设计思想: <https://yuque.antfin-inc.com/tmf/documents/design-concept> 星环概念解读: <https://yuque.antfin-inc.com/wdyfa7/imf6lm/obr4et>

商业能力样例: <https://yuque.antfin-inc.com/tmf/documents/qf24eb>

域能力和商业能力的关系: <https://yuque.antfin-inc.com/onetms/gzhy63/wim3p8>

为什么业务定制要基于商业能力: <https://ata.alibaba-inc.com/articles/141009>

商业能力介绍 [https://ata.alibaba-inc.com/articles/166566?
spm=ata.23639746.0.0.570c363cVa9u6y](https://ata.alibaba-inc.com/articles/166566?spm=ata.23639746.0.0.570c363cVa9u6y)